

Vero: Can AI Agents Build Formally Verified Software Repositories?

Zhe Ye^{1*} Hantao Lou^{1*} Yuechun Sun^{2*} Peiyang Song³ Zhengxu Yan⁴ Timothe Kasriel¹

Qingyang Zhang¹ Kaiyu Yang⁵ Soonho Kong^{6†} Jingxuan He¹ Dawn Song¹

¹UC Berkeley ²University of Chicago ³California Institute of Technology
⁴Stanford University ⁵Apodex ⁶Amazon Web Services

Abstract

AI agents are increasingly used for programming, but do not provide any guarantee on the correctness of generated code. Verified code generation, in which an agent produces both an implementation and a machine-checked proof of its specification, offers a stronger path toward trustworthy AI-generated software. Existing benchmarks in this direction either focus on individual functions or only evaluate proof generation with provided implementations. It is still an open question whether agents can make coherent implementation and proof choices across real multi-module codebases. To bridge this gap, we introduce Vero, the first benchmark to evaluate joint implementation and proof synthesis at the repository level. Vero contains 43 multi-module instances sourced from real-world repositories spanning Python, Dafny, Verus, and Coq, and covering diverse domains from cryptographic protocols to distributed systems. Each instance consists of a multi-module Lean 4 repository with predetermined API interfaces, manually curated formal specifications, and reference implementations, supporting both proof-only and code-and-proof evaluation modes. To improve benchmark reliability, Vero also includes an audit mechanism where agents are allowed to formally prove unsatisfiability of provided specification or incorrectness of reference code, which surfaces and corrects latent code and specification errors during curation. We evaluate frontier coding-agent configurations with Lean toolchain access. The strongest agent fully solves only 27 of 43 instances and closes no specifications on the hardest repositories. Vero provides a concrete testbed for measuring progress toward repository-scale verified software synthesis, where current agents still fall short. We release the benchmark, curation pipeline, and evaluation harness at <https://github.com/sunblaze-ucb/vero>.

1 Introduction

AI agents are increasingly used in software engineering tasks [15, 34]. The correctness of agent-produced code, however, is typically assessed through unit tests and human review. These methods catch anticipated failures but cannot rule out edge-case bugs and security vulnerabilities, which are most damaging in protocol and infrastructure software. Formal verification offers a stronger alternative: a machine-checked proof that an implementation satisfies its specification rules out, for all inputs, every class of bug the specification captures. Therefore, it is critical to rigorously assess whether AI agents can perform formal verification at the scale of real-world software.

*Equal contribution.

†Work does not relate to position at Amazon.

Existing benchmarks for verified code generation either target individual functions [20, 27, 8, 36, 28, 4] or evaluate only proof generation with a fixed implementation [39, 31, 33, 25], leaving the joint code-and-proof task unaddressed. Repository-scale verified code generation does not reduce to scaling up function-level techniques, because code, specifications, and proofs are deeply interdependent across files: a proof for one function may depend on lemmas about lower-level utilities, and revising an implementation can invalidate proofs throughout the repository. Agents must reason globally about the consistency of the entire codebase, rather than locally about a single function. This is also where verified code generation matters in practice, since real-world verified software, from operating system kernels [16, 11] to cryptographic protocols [23, 9] and distributed systems [12], lives in repositories rather than standalone functions. Closing this evaluation gap is essential for measuring and driving progress on agents that can produce verified software at the scale for practical impact.

To bridge this gap, we introduce Vero, the first repository-level benchmark for verified code generation in Lean 4 [6]. Vero consists of 43 benchmark instances curated from real-world repositories spanning a diverse set of source languages, including Python, Dafny [19], Verus [18], and Coq [3], and covering a range of domains and difficulty levels from foundational data structures to verified systems. Sourcing from verification-aware languages allows us to leverage existing formal specifications and proofs as ground truth, while Python repositories expand domain coverage to widely used algorithmic and systems code. Each instance is a Lean 4 project comprising multiple modules with their data type definitions, auxiliary definitions, *API signatures*, and human-curated ground-truth *specifications*. In total, Vero contains 743 scored APIs and 2,705 specifications, so the 43 repositories correspond to thousands of verification obligations rather than 43 standalone tasks. We constructed benchmark instances through a multi-stage curation pipeline where every translated definition, specification, and proof obligation is manually reviewed and validated by the authors. The agent’s task is to fit into this scaffold by discharging two types of obligations: an *implementation obligation* for each API signature, and a *proof obligation* for each specification. Vero supports two task modes. In the primary *code-and-proof* mode, agents must discharge both obligations: synthesize implementations for every API and machine-checked Lean 4 proofs that they satisfy all specifications. In the *proof-only* mode, reference implementations are additionally provided and the agent discharges only the proof obligations. Vero is, to our knowledge, the first benchmark that evaluates agents on joint code-and-proof generation at the repository level.

A persistent challenge in constructing formal-verification benchmarks is that even carefully curated specifications and reference implementations can contain subtle errors, a problem documented across formal verification projects and benchmarks [10]. Such errors would silently penalize otherwise capable agents and treat benchmark defects as agent failures. Vero addresses this through a novel audit mechanism. Rather than treating such errors as unexplained agent failures, the mechanism accepts formal proofs of specification unsatisfiability or reference implementation incorrectness to flag the affected instance for curator review, and uses the formal witnesses to guide correction. During curation, the audit mechanism surfaced several latent errors that escaped manual review, and helped guide the benchmark improvement by providing formal evidences like counter examples.

We evaluate four frontier model configurations under two coding-agent harnesses on Vero across both task modes. Vero is frontier-resistant: the best-performing configuration fully solves only 27 of 43 instances in code-and-proof, and 10 instances resist every configuration in both modes. Our analysis further reveals that the unsolved instances concentrate on specifications encoding cross-module invariants, protocol consistency, and custom mathematical theories, where current agents attempt local proofs of individual specifications rather than building the reusable lemma libraries that verification at this scale requires. We further find that implementation freedom in code-and-proof cuts both ways. Agents can sidestep hard-to-verify reference algorithms by writing simpler implementations that satisfy the same specifications. On harder instances, however, agents fail to coordinate the two artifacts, producing rewrites that break existing proofs or introduce build errors elsewhere in the repository. Together these findings provide concrete directions for advancing repository-scale verified code generation.

Contributions. In summary, we make the following key contributions:

- We introduce Vero, the first repository-scale benchmark for verified code generation in Lean 4, comprising 43 instances curated from real-world repositories across diverse source languages and domains, together with a semi-automated, multi-language curation pipeline that supports its construction and future extension.

- We design a formal audit mechanism that accepts machine-checked proofs of specification unsatisfiability or reference implementation incorrectness, turning latent benchmark errors into actionable findings and enabling continuous quality improvement as agents grow more capable.
- We conduct a comprehensive evaluation of state-of-the-art coding agents and frontier LLMs on Vero across two task modes, providing rigorous baselines and actionable insights toward scalable, automated software verification.

2 Background and Related Work

Existing benchmarks for verified code generation share several limitations that constrain their utility for evaluating coding agents in realistic settings.

First, most benchmarks evaluate verified code generation only at the level of individual functions. In Lean 4, miniCodeProps [20], FVAPPS [8], VERINA [36], and CLEVER [28] all target standalone tasks drawn from algorithmic problem sets such as HumanEval [5], MBPP [2], and APPS [13]. The same pattern holds in benchmarks for SMT-based verifiers such as Dafny [19] and Verus [18]: DafnyBench [21], VerusBench [32], AlgoVeri [38], VeriCoding [4], VerifyThisBench [7], and VeriEquivBench [37] all operate on individual algorithms or methods. While these benchmarks have driven measurable progress on isolated tasks, they cannot capture the cross-module dependencies and long-horizon reasoning that real-world verified codebases require. Vero addresses this gap by curating multi-module Lean 4 projects with type definitions, API signatures, and specifications spread across files, requiring agents to reason about the consistency of the entire repository.

Second, the few existing repository-scale benchmarks evaluate proof generation alone, leaving the joint code-and-proof task unaddressed. RVBench [39] provides 755 proof completion tasks across four Verus projects; VeruSAGE-Bench [33] contains 849 proof tasks extracted from eight Verus-verified Rust systems; and VeriSoftBench [31] packages 500 Lean 4 proof obligations from open-source formal-methods developments. For Coq, CoqStoq [29] offers a large corpus of repository-level theorems for proof synthesis, while case studies such as FSCQ proof generation [24] target proof completion in verified system software. All these benchmarks provide reference implementations and evaluate only the agent’s ability to discharge proof obligations. This setting omits a fundamental challenge of repository-scale verified code generation, since the choice of implementation directly affects which proofs are tractable and the two artifacts must be coherent across the entire repository. Vero addresses this by supporting both proof-only and code-and-proof modes, with the latter requiring agents to jointly synthesize implementations and machine-checked proofs across the repository, exposing the implementation-proof coupling that prior repository-scale benchmarks abstract away. To our knowledge, Vero is the first benchmark to evaluate joint code-and-proof generation at the repository level.

Third, prior benchmarks are limited in scope and face contamination risks. A large body of Lean 4 benchmarks targets formalized mathematics rather than software, including LeanDojo [35], LeanAgent [17], and APE-Bench [30], all built on the Mathlib library [22]. Although these benchmarks have driven impressive progress in neural theorem proving, the structure of mathematical proof differs substantially from the verification of imperative software, where reasoning is coupled to executable definitions, data types, and effectful behavior. A separate concern is the contamination of training data, since reference solutions for many widely used program sources are freely available online and may have been included in the pretraining data of evaluated models [14, 26]. Benchmarks that directly reuse such sources inherit this exposure when the underlying problems and their reference solutions remain widely accessible. Vero addresses this by sourcing tasks from real-world repositories across multiple source languages and producing novel Lean 4 formalizations through manual curation, so that no Lean 4 ground-truth proof for any benchmark instance is directly available. This translation from source languages to Lean 4 is itself novel curation work, providing a structural contamination guarantee that complements the diversity of domains and difficulty levels in the corpus.

Finally, prior benchmarks rarely consider evaluation protocols designed for agent-based settings. Most existing benchmarks evaluate raw LLM outputs in single-shot or simple iterative settings, without the tool access that deployed coding agents use in practice. In contrast, Vero evaluates frontier coding agents with full tool access. Furthermore, prior benchmarks provide no mechanism to detect or correct errors in their own ground truth, a concern recently surfaced in formal verification benchmarks [10]. When a specification is wrong or a reference implementation is incorrect, it

becomes impossible to distinguish agent failures from benchmark defects. Prior repository-scale benchmarks also lack anti-cheating measures, leaving open whether reported results reflect genuine verification capability or reward hacking. Vero addresses these through a formal audit mechanism that accepts machine-checked negative evidence (Section 3.5) and anti-cheating safeguards that prevent reward hacking through axiom injection or definition edits (Section 3.2).

3 The Vero Benchmark

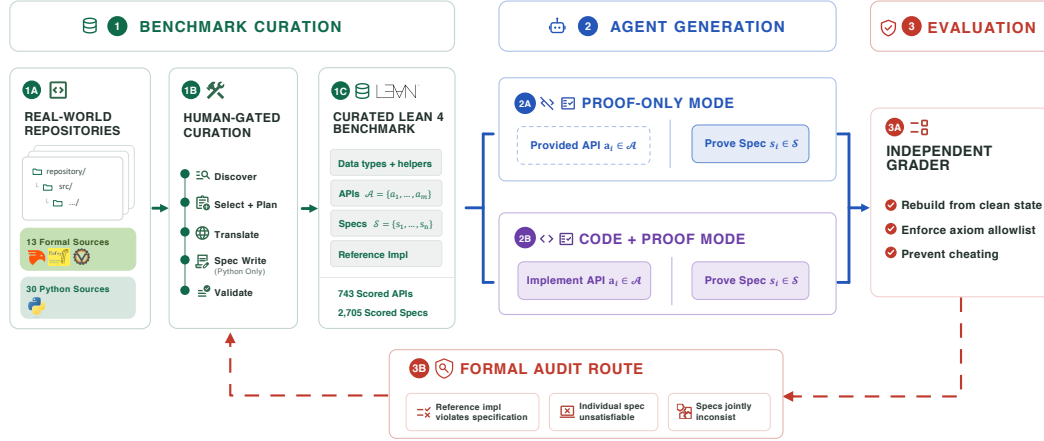


Figure 1: **Vero’s end-to-end construction and evaluation workflow.** Human-gated curation converts real-world Python and formal-language repositories into Lean 4 benchmark instances with fixed definitions, API signatures, specifications, and reference implementations. Agents are evaluated in proof-only or code-and-proof mode and scored by an independent grader, while the formal audit route returns machine-checked evidence of benchmark defects to curation.

Figure 1 summarizes Vero from benchmark curation to agent evaluation. We first define the instance format (Section 3.1) and two task modes (Section 3.2), then describe the curation pipeline (Section 3.3) and resulting dataset (Section 3.4), and finally introduce the formal audit mechanism (Section 3.5).

3.1 Data Format

A Vero instance is a multi-module Lean 4 project where the curator provides three layers of fixed content, namely data type and helper definitions, API signatures, and formal specifications. The agent’s task is to discharge two kinds of obligations within this scaffold, an *implementation obligation* for each API signature and a *proof obligation* for each specification. Formally, a Vero instance contains following components:

1. A collection of **data type and helper definitions** shared across the rest of the instance.
2. A set of **API signatures** $\mathcal{A} = \{a_1, \dots, a_m\}$, each with a reference implementation.
3. An **interface structure type** `RepoImpl` that collects all required API implementations, with one field of type a_i for each $a_i \in \mathcal{A}$.
4. A set of **specifications** $\mathcal{S} = \{S_1, \dots, S_n\}$, each of type `RepoImpl` $\rightarrow$ `Prop`.
5. A **canonical implementation** `canonical` : `RepoImpl`, populated by the reference implementations in proof-only mode and filled in by the agent in code-and-proof mode.

Crucially, each specification is parameterized over `RepoImpl` instead of fixed to a single implementation. This design allows flexible switch of the implementation target to prove, and is the key to support the audit mechanism explained in Section 3.5. Figure 2 illustrates the format on a small example drawn from the a reference instance.

```

1  -- API signatures
2  abbrev CreateAccountSig := AccountId -> Ledger -> Ledger
3  abbrev AccountExistsSig := AccountId -> Ledger -> Bool
4  abbrev GetBalanceSig    := AccountId -> Ledger -> Option Balance
5
6  -- Interface structure: one field per API signature
7  structure RepoImpl where
8    createAccount : CreateAccountSig
9    accountExists : AccountExistsSig
10   getBalance    : GetBalanceSig
11
12  -- Canonical implementation (instantiated to reference implementation in proof-
    only mode and agent solution in code-and-proof mode)
13  def canonical : RepoImpl := {
14    createAccount := Bank.createAccount,
15    accountExists := Bank.accountExists,
16    getBalance    := Bank.getBalance,
17  }
18
19  -- Specification: a predicate over implementations
20  def spec_create_zero_balance (impl : RepoImpl) : Prop :=
21    ∀ (id : AccountId) (ledger : Ledger),
22      impl.accountExists id ledger = false ->
23      impl.getBalance id (impl.createAccount id ledger) = some 0
24
25  -- Theorem: Proof obligation on the canonical implementation
26  theorem proof_create_zero_balance : spec_create_zero_balance canonical := by
    sorry

```

Figure 2: Excerpt from a Vero example illustrating a benchmark instance. Frozen content like API signatures and specifications are provided by the curator. The agent fills in the body of `canonical` (in code-and-proof mode) and the proof obligation marked `sorry` (in both mode).

3.2 Task and Evaluation

Vero supports two task modes that differ in which obligations the agent must discharge. In both modes, we evaluate whether the agent can produce artifacts that prove all specifications for a given Vero instance. Full coverage is necessary because any unproven specification leaves room for a bug it would have caught, and because specifications vary widely in difficulty, so partial coverage can be inflated by easy ones.

Proof-only mode. In this mode, we measure whether the agent can prove every specification against the reference implementation, and `canonical` is instantiated from reference implementation. Formally, for each specification $S_i \in \mathcal{S}$, the agent must produce a machine-checked Lean 4 proof of $S_i(\text{canonical})$.

Code-and-proof mode. In this mode, we measure whether the agent can both implement every API and prove every specification against its own implementations. `canonical` in this mode is instantiated from the agent’s own implementations. Formally, the agent must both provide a function body of type a_i for each $a_i \in \mathcal{A}$ and produce a machine-checked Lean 4 proof of $S_i(\text{canonical})$ for each $S_i \in \mathcal{S}$.

Code-and-proof is qualitatively different. Vero is, to our knowledge, the first repository-scale benchmark to require joint implementation and proof synthesis. Existing repository-scale benchmarks fix the implementation and evaluate proof generation in isolation, ignoring how implementation and proof choices interact in real verification work. The agent can refactor implementations toward proof-friendly forms, but implementation changes can also invalidate proofs across modules. The two modes therefore measure qualitatively different yet complementary capabilities of formal verification.

Preventing reward hacking. To ensure the benchmark evaluates genuine verification work, the grader prevents both edits to benchmark content and proof-trivializing mechanisms. First, we use explicit markers to constrain the regions agents may modify. The grader extracts only contents from these regions and inserts them into a fresh copy of the source benchmark before grading. Second, an axiom allowlist rejects proofs that depend on agent-introduced or untrusted axioms. Finally, a rule-based detector and an LLM judge screen agent-introduced declarations for mechanisms that can trivialize proofs or decouple logical semantics from executable behavior, including malicious typeclass instances and noncomputable choice combined with `@[implemented_by]` (e.g., through `Exists.choose`). We discuss these anti-cheating mechanisms in Appendix D.

3.3 Vero Curation Pipeline

To construct Vero, we developed a multi-stage pipeline that converts source repositories into Vero instances. Each stage runs as an LLM agent and is reviewed by a human curator before the next stage begins. The pipeline supports two source tracks, distinguished by the kind of curation work each repository requires. *Track 1* covers repositories already written in a formal language such as Dafny, Verus, or Coq, where we need to translate existing types, definitions, and specifications into Lean 4. *Track 2* covers non-formal repositories, typically Python, where we translate each implementation into Lean 4 and also write the specifications based on original repositories and documentations. The curation pipeline and human reviewers ensure that the resulting Lean projects form well-defined benchmark instances while remaining faithful to the structure and intended behavior of the source repositories.

Repository selection. We select repositories using four criteria: real-world provenance, domain diversity, a wide range of scale and difficulty, and feasibility of faithful translation into a Lean scaffold of moderate size. Track 1 draws from actively used verification projects, while Track 2 draws from high-star, widely used Python libraries, together spanning smart contracts and blockchain protocols, distributed systems, security-critical parsing and encoding, formalized mathematical algorithms, and core data structures and algorithms.

Pipeline stages. The pipeline runs the following stages, and each stage requires curator approval before proceeding. The first three stages, *discover*, *select*, and *plan*, list all available declarations from the source repository, identify which ones to include, compute their dependencies, and commit to specific Lean signatures and a module layout. The *translate* stage orchestrates per-module translations, dispatching each module to an executor agent in dependency order to enable parallel and scoped translation. The curator reviews each translated definition, and items that fail review are re-translated. On Track 2, an additional *spec writing* stage drafts specifications in natural language for curator review and then formalizes the approved set in Lean. We require specifications in Track 2 are grounded in the upstream documented behavior and test suites and manually reviewed for semantic fidelity, and ensure that each API is constrained by multiple interacting specifications. The final *validate* stage runs automated structure and build checks alongside LLM-judged semantic reviews of specification intent, idiomatic Lean usage, and other aspects.

Reusability. We build the curation pipeline stages by creating modular agent skills [1], including source-language translation rules, per-stage agent behaviors, and validation prompts. We currently provide source-language skills for Python, Dafny, Verus, and Coq. Because the pipeline stages, validation logic, and Lean output format are shared across tracks, adding a new source language requires only a new skill. This makes Vero directly extensible to more source languages and verification domains.

3.4 Dataset Overview

Vero consists of 43 instances curated from real-world repositories, with 13 from Track 1 (formal) and 30 from Track 2 (Python). The evaluated corpus totals 743 scored APIs and 2,705 scored specifications. Table 1 reports per-track summary statistics.

The instances span a broad range of verification-relevant code, including smart contracts and blockchain protocols, distributed systems and consensus, security-critical infrastructure, formal-mathematics libraries, data structures, algorithm collections, and numerical utilities. They also vary

Table 1: **Summary statistics of Vero by source track.** We report the per-instance mean and maximum for each metric. Source LoC is the lines of valid code of upstream repositories selected for curation, excluding blank lines and comments.

Track	Source languages	Inst.	APIs		Specs		Source LoC	
			mean	max	mean	max	mean	max
Track 1 (formal)	Dafny, Verus, Coq	13	36.0	88	92.8	203	7,759	56,887
Track 2 (non-formal)	Python	30	9.2	71	50.0	109	793	4,047
Overall		43	17.3	88	62.9	203	2,899	56,887

widely in scale, from a few hundred to over fifty thousand source lines per repository and from single digits to over 100 specifications per instance, reflecting a corresponding range of verification difficulty. This diversity across domains and difficulty levels supports comprehensive assessment of an agent’s formal-verification capability.

Contamination. A central concern for benchmarks built on public code is contamination, the risk that an evaluated model has memorized the reference solutions during pretraining. Vero is largely free of this concern by construction. The Lean 4 specifications, implementations, and proofs are all novel curation work with no public Lean 4 ground truth. Track 1 upstream content exists only in other formal languages, and Track 2 upstream content is Python without specifications or proofs.

3.5 Benchmark Formal Audit Mechanism

Even carefully curated formal-verification benchmarks can contain latent errors, including specifications that no implementation satisfies, reference implementations that fail their intended specifications, and specification sets that are mutually inconsistent. Such errors are subtle and can survive type-checking, builds, and curator review, but they can surface when an agent attempts to prove. Treating these silent failures as agent errors conflates benchmark defects with agent capability.

Vero addresses this with an *audit mechanism*, a separate formal-evidence path that accepts three forms of machine-checked negative evidence:

$$\neg \bigwedge_{S \in \mathcal{S}} S(\text{canonical}), \quad (1)$$

$$\neg \exists \text{impl} : \text{RepoImpl}, S(\text{impl}) \text{ for some } S \in \mathcal{S}, \quad (2)$$

$$\forall S \in \mathcal{S}', \exists \text{impl} : \text{RepoImpl}, S(\text{impl}) \text{ with } \neg \exists \text{impl}, \bigwedge_{S \in \mathcal{S}'} S(\text{impl}) \text{ for some } \mathcal{S}' \subseteq \mathcal{S}. \quad (3)$$

Equation (1) catches reference implementation bugs in proof-only mode. In code-and-proof mode, Equation (2) catches individually unsatisfiable specifications S , and Equation (3) catches inconsistent specification sets while preserving evidence that each member remains individually satisfiable.

During benchmark curation, we used the audit mechanism alongside evaluated agents to iteratively improve benchmark quality. We present details case studies on how this mechanism improves the benchmark quality in Appendix F. The audit mechanism is also valuable in supporting continuous benchmark improvement as future agents grow more capable.

4 Evaluation

In this section, we evaluate four frontier coding-agent configurations on Vero under both task modes and analyze where current agents succeed and where they fall short.

4.1 Experimental Setup

We evaluate two coding-agent harnesses with frontier models. Codex (v0.140.0) is paired with two GPT-5.5 profiles: the default reasoning effort (medium) and the xhigh reasoning effort. Claude Code (v2.1.191) is paired with Claude Opus 4.8 and Claude Sonnet 5 at xhigh reasoning effort. All agents run with full tool access, including file-system edits, build invocations, and the Lean toolchain. We report the count of fully solved instances in both code-and-proof and proof-only modes. We

adopt full solve rather than partial specification coverage because partial coverage can be inflated by easy specifications, and in code-and-proof mode an unproven specification may indicate not merely a missing proof but an implementation that violates it, so only full coverage certifies the agent’s code as correct. We use Lean v4.29.1 throughout the evaluation. The detailed setup is described in Appendix A.

4.2 Evaluation Results and Analysis

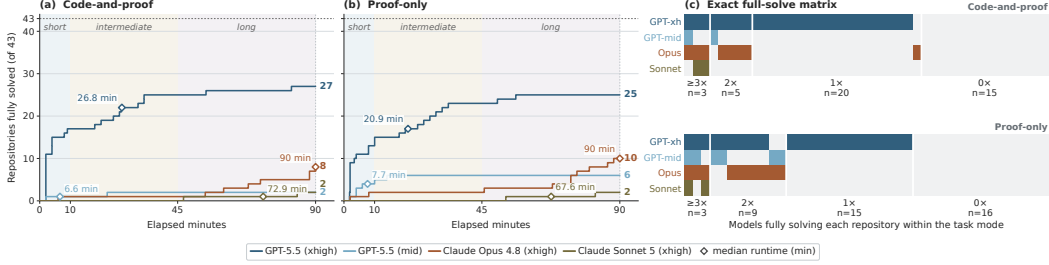


Figure 3: **Agent performance on Vero.** (a,b) Cumulative full solves, out of 43, over the 90-minute budget in code-and-proof and proof-only; diamonds mark each configuration’s median runtime. (c) Which configurations fully solve each instance in each mode.

GPT-5.5 (xhigh) leads both modes, but Vero remains frontier-resistant. GPT-5.5 (xhigh) fully solves 27 of 43 instances in code-and-proof and 25 in proof-only, ahead of Claude Opus 4.8 (8 and 10), GPT-5.5 (mid) (2 and 6), and Claude Sonnet 5 (2 in each mode) (Figure 3(a,b)). It also progresses fastest, reaching 25 and 23 full solves within 45 minutes. Nevertheless, 10 instances resist all eight configurations across both modes, and most solved instances are closed by only one configuration (Figure 3(c)). The bottleneck is not local proof skill: GPT-5.5 (xhigh) passes 87.3% of specifications in code-and-proof and 85.8% in proof-only. High per-specification coverage therefore does not imply repository completion. The specifications left are mostly hardest to prove, where agents must discover shared invariants, organize them into a reusable lemma library, and preserve that library in a buildable artifact.

Implementation freedom matters through the proof target; proof volume is what separates full solves. Across the 172 matched instance–agent pairs, 26 are fully solved in both modes, 13 only in code-and-proof, 17 only in proof-only, and 116 in neither (Figure 4(a)). We manually examined agent solutions in code-and-proof, and found five instance-agent pairs across three repositories where the agent replaces a hard-to-verify reference algorithm with a simpler implementation of its own that satisfies the same specifications (Appendix G). Each substitution is behaviorally correct rather than a shortcut, so what the agent gains is provability and what it gives up is efficiency. Across these five pairs the agent closes all 250 specifications, whereas proving against the fixed reference in proof-only closes only 201. Conversely, we find 17 pairs that are full solves in proof-only but not in code-and-proof, where we see implementation adds difficulty for models. The size statistics in Figure 4(b–d) show that fully solved and unfinished runs write similar amounts of implementation code, while full solves contain roughly twice the proof text and correspondingly higher proof-to-implementation ratios. Completing a repository-level formal verification is a matter of sustained proof work rather than implementation volume.

Completed repositories rely on shared lemma libraries. Across the 82 full solves, agent-written helper theorems contain a median of 73.6% of proof lines in code-and-proof and 71.6% in proof-only (Figure 5(b)). These lemmas are not for individual proof goal but serve as shared libraries, as 80 of the 82 solves share a helper across at least two specifications and 65 across at least five (Figure 5(a)). Completed repositories are organized around reusable lemma libraries rather than independent proofs per specification.

Deep lemma chains predict where other agents fail. We measure each specification’s helper-chain depth in one GPT-5.5 (xhigh) proof-only full solve and check how the same specifications fare in the

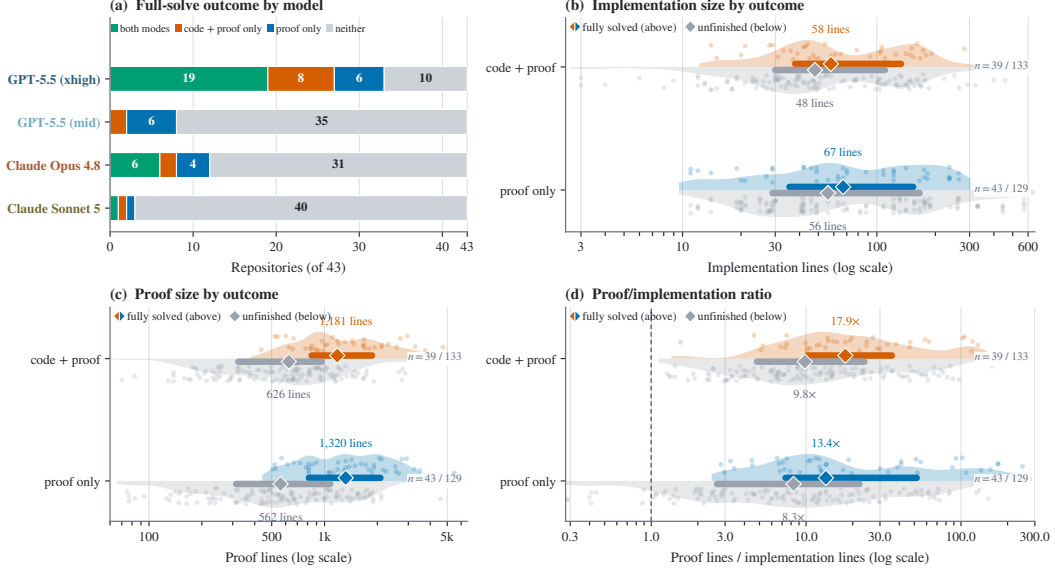


Figure 4: **Full-repository outcomes and artifact sizes by task mode.** (a) Paired full-solve outcomes across the two modes for each of the 172 instance-agent pairs. (b–d) Implementation lines, proof lines, and their ratio at the end of each of the 344 runs, split by mode and by whether the run is a full solve. Diamonds mark medians and thick segments mark interquartile ranges.

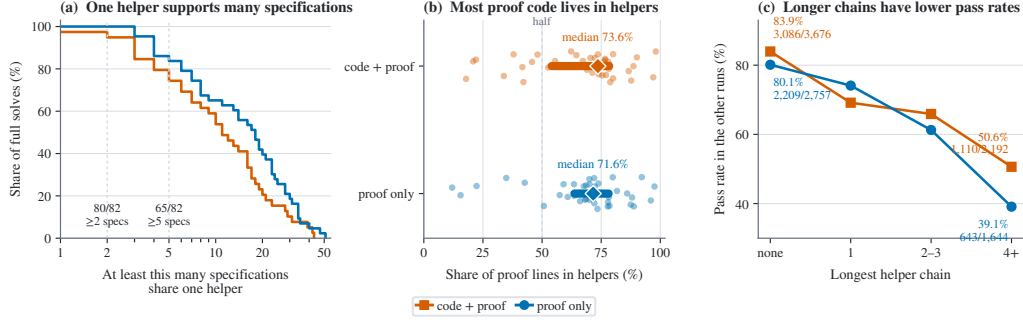


Figure 5: **Proof structure in the 82 full solves.** A helper is a theorem the agent writes to support its specification proofs. (a) How many specifications share one helper. (b) Share of proof lines that live in helpers. (c) Pass rates in the other seven runs, grouped by each specification’s helper-chain depth in GPT-5.5 (xhigh)’s proof-only solutions.

other seven runs (Figure 5(c)). Specifications needing no helper pass at 83.9% in code-and-proof and 80.1% in proof-only, falling to 50.6% and 39.1% at depth four or more. Deep specifications are hard because they require layered and global reasoning, where the agent must first build a chain of prerequisite lemmas about lower-level definitions.

Agents commit to an implementation early and spend the rest of the budget on proofs. Every configuration reaches its final median implementation size within the first half of the run, while proof text keeps growing until the deadline (Figure 6). GPT-5.5 (xhigh) fixes its median 65 implementation lines by minute 30 and then grows proof text from 883 to 1,077 lines, and Claude Sonnet 5 adds only 5 implementation lines after minute 60 while its proof text nearly doubles. The agents differ mainly in pace, as GPT-5.5 (xhigh) plateaus early while Claude Opus 4.8 and Sonnet 5 are still writing proofs when time expires, suggesting their lower solve counts partly reflect the time budget. The one-way ordering also shows that agents treat the implementation as fixed scaffolding rather than a lever, so

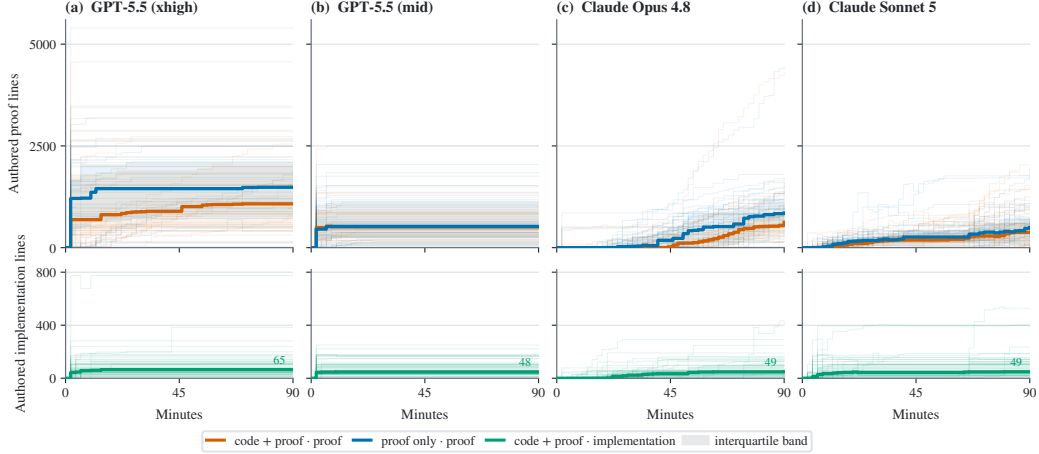


Figure 6: **Agents fix implementations early and grow proofs until the deadline.** Columns are agent configurations. Top row shows authored proof lines over time in both modes; bottom row shows authored implementation lines in code-and-proof. Thin lines are per-repository trajectories, bold lines are medians, and bands are interquartile ranges. Counts exclude blank lines, comments, and placeholders.

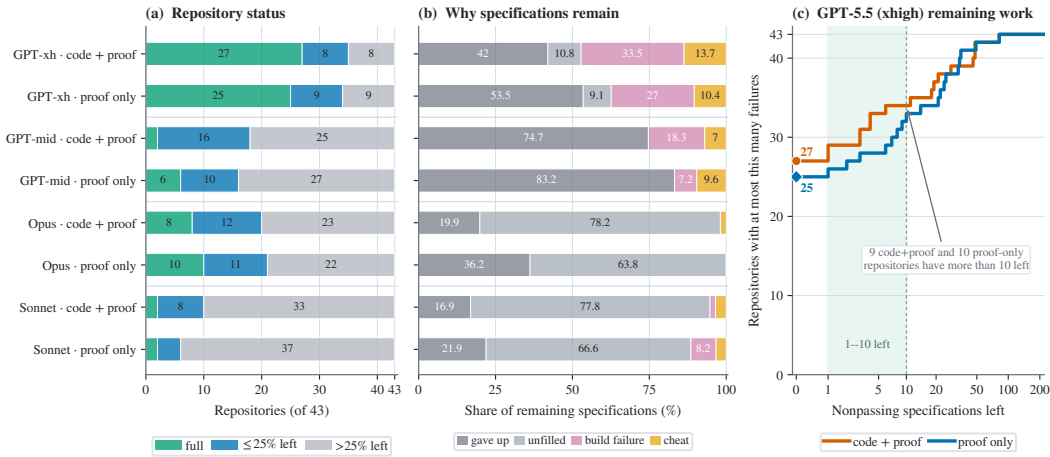


Figure 7: **Where failures remain at the end of a run.** (a) Repositories grouped by the share of their specifications still failing. (b) Breakdown of remaining specifications by failure reason. (c) Number of GPT-5.5 (xhigh) repositories with at most x failing specifications.

when proofs get stuck they grind on the proof layer instead of returning to refactor the code into a more provable form.

Strong agents attempt the hardest obligations but cannot close them. GPT-5.5 (xhigh) ends with at most a quarter of each repository’s specifications failing in 35 of 43 code-and-proof repositories and 34 of 43 in proof-only, more than any other configuration (Figure 7(a)). The failure reasons also differ by strength: a third of its remaining specifications fail at build time and a further 14% are rejected as cheating, whereas Claude Opus 4.8 and Claude Sonnet 5 leave roughly 78% of theirs with no proof body at all (Figure 7(b)). Strong agents thus fail while attempting the hardest obligations, whereas weaker agents never reach them. The remaining failures are not near misses, since only 6 of its 16 unfinished code-and-proof repositories are within five specifications of completion and nine have more than ten left (Figure 7(c)). The residual specifications typically form a cluster that shares a few missing invariants, so closing them requires the inductive generalization the agent never found, not additional proof attempts.

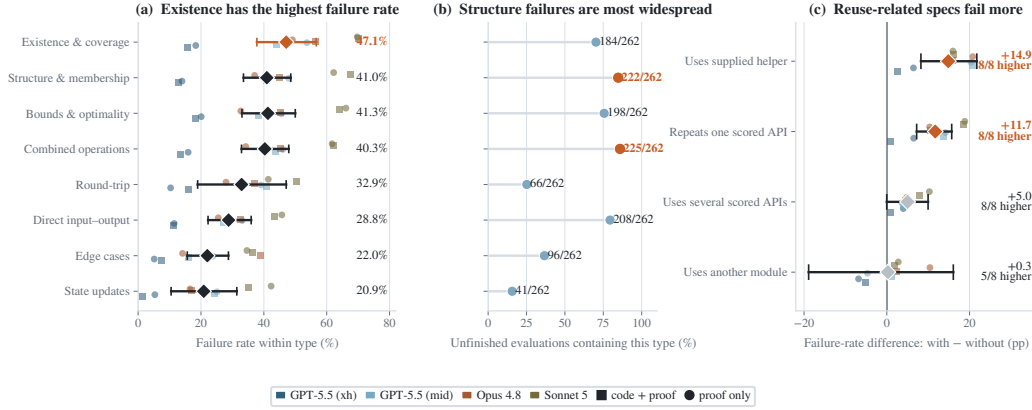


Figure 8: **Which specifications fail.** (a) Failure rate by semantic type, equalized across repositories. (b) How often each type appears among the 262 unfinished evaluations. (c) Within-repository failure-rate differences for four statement features. Diamonds and whiskers show aggregate estimates with bootstrap 95% confidence intervals; small markers show the eight agent and mode configurations. Types and features may overlap.

Specifications about global properties and definition reuse fail most. Existence-and-coverage specifications, which assert that an output covers or accounts for all relevant inputs, have the highest failure rate at 47.1% and stay highest under every sensitivity analysis (Figure 8(a)). These are global properties that cannot be closed by case-splitting on a single call, matching the inductive-reasoning gap identified above. Within repositories, specifications that use a supplied helper definition or call one API repeatedly fail 14.9 and 11.7 points more often, consistently across all eight configurations, while merely referencing several APIs or another module adds little (Figure 8(c)). Difficulty therefore comes from unfolding and reasoning about supplied definitions and iterated behavior, not from crossing module boundaries.

5 Conclusion

We introduced Vero, the first benchmark for joint implementation and proof synthesis at the repository level in Lean 4. Vero comprises 43 multi-module Lean 4 instances curated from real-world repositories in Dafny, Verus, Coq, and Python, together with an extensible curation pipeline, an evaluation harness for both task modes, and a formal audit mechanism that turns machine-checked negative evidence into benchmark corrections. Our evaluation shows that Vero is frontier-resistant, as the strongest configuration fully solves only 27 of 43 instances in code-and-proof mode and 10 instances resist every configuration in both modes. The gap is not local proof skill but repository-scale organization. The strongest agent pass over 80% of individual specifications yet fails to discover shared invariants, build the reusable lemma libraries that deep specifications require, and keep the whole repository consistent and buildable. We hope Vero serves as a concrete testbed for driving agents toward producing fully verified software at practical scale.

Limitations. We discuss the following limitations. First, Vero currently targets Lean 4 only. We chose Lean as the initial target because it is a particularly active ecosystem for LLM-based formal reasoning, providing a favorable setting for current agents. At the same time, Track 1 translates projects from Dafny, Verus, and Coq, so their specification and invariant structures are represented in the benchmark, and the curation pipeline is extensible to other target languages (Section 3.3). Second, the corpus favors code that translates cleanly into Lean, and the main absent class is concurrent or temporal protocols whose upstream formalizations do not port to a Lean scaffold of moderate size. Benchmarking formal verification of incremental maintenance tasks is also a valuable future direction. Finally, the audit mechanism certifies formal satisfiability but cannot ensure that specifications are semantically correct or complete, so all specifications are manually reviewed and Track 1 specifications are additionally cross-checked against the source-language formalizations.

Acknowledgments and Disclosure of Funding

This work is partially supported by a grant from Amazon. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of Amazon.

References

- [1] Anthropic. Agent skills. <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>, 2026. Accessed: 2026-05-02.
- [2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [3] Bruno Barras, Samuel Boutin, Cristina Cornes, Judicaël Courant, Jean-Christophe Filliatre, Eduardo Gimenez, Hugo Herbelin, Gerard Huet, Cesar Munoz, Chetan Murthy, et al. *The Coq proof assistant reference manual: Version 6.1*. PhD thesis, Inria, 1997.
- [4] Sergiu Bursuc, Theodore Ehrenborg, Shaowei Lin, Lacramioara Astefanoaei, Ionel Emil-ian Chiosa, Jure Kukovec, Alok Singh, Oliver Butterley, Adem Bizid, Quinn Dougherty, et al. A benchmark for vericoding: formally verified program synthesis. *arXiv preprint arXiv:2509.22908*, 2025.
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [6] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris Van Doorn, and Jakob von Raumer. The Lean theorem prover (system description). In *International Conference on Automated Deduction (CADE)*, 2015.
- [7] Xun Deng, Sicheng Zhong, Barış Bayazit, Andreas Veneris, Fan Long, and Xujie Si. Verifythis-bench: Generating code, specifications, and proofs all at once. *arXiv preprint arXiv:2505.19271*, 2025.
- [8] Quinn Dougherty and Ronak Mehta. Proving the coding interview: A benchmark for formally verified code generation. *arXiv preprint arXiv:2502.05714*, 2025.
- [9] Andres Erbsen, Jade Philipoom, Jason Gross, Robert H. Sloan, and Adam Chlipala. Simple high-level code for cryptographic arithmetic - with proofs, without compromises. *2019 IEEE Symposium on Security and Privacy (SP)*, pages 1202–1219, 2019.
- [10] Yueyang Feng, Dipesh Kafle, Vladimir Gladstein, Vitaly Kurin, George Pîrlea, Qiyuan Zhao, Peter Müller, and Ilya Sergey. Certified program synthesis with a multi-modal verifier. *arXiv preprint arXiv:2604.16584*, 2026.
- [11] Ronghui Gu, Zhong Shao, Hao Chen, Xiongnan Newman Wu, Jieung Kim, Vilhelm Sjöberg, and David Costanzo. CertiKOS: An extensible architecture for building certified concurrent OS kernels. In *Symposium on Operating Systems Design and Implementation (OSDI)*, 2016.
- [12] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath T. V. Setty, and Brian Zill. Ironfleet: proving practical distributed systems correct. *Proceedings of the 25th Symposium on Operating Systems Principles*, 2015.
- [13] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. Measuring coding challenge competence with APPS. In *Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2021.
- [14] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

- [15] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [16] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an OS kernel. In *Symposium on Operating systems principles (SOSP)*, 2009.
- [17] Adarsh Kumarappan, Mo Tiwari, Peiyang Song, Robert Joseph George, Chaowei Xiao, and Anima Anandkumar. Leanagent: Lifelong learning for formal theorem proving. *arXiv preprint arXiv:2410.06209*, 2024.
- [18] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying rust programs using linear ghost types. *Proceedings of the ACM on Programming Languages*, 2023.
- [19] K Rustan M Leino. Dafny: An automatic program verifier for functional correctness. In *International Conference on Logic for Programming Artificial Intelligence and Reasoning (LPAR)*, 2010.
- [20] Evan Lohn and Sean Welleck. miniCodeProps: a minimal benchmark for proving code properties. *arXiv preprint arXiv:2406.11915*, 2024.
- [21] Chloe Loughridge, Qinyi Sun, Seth Ahrenbach, Federico Cassano, Chuyue Sun, Ying Sheng, Anish Mudide, Md Rakib Hossain Misu, Nada Amin, and Max Tegmark. DafnyBench: A benchmark for formal software verification. *Transactions on Machine Learning Research*, 2025.
- [22] Mathlib community. The Lean mathematical library. In *Certified Programs and Proofs (CPP)*, 2020.
- [23] Marina Polubelova, Karthikeyan Bhargavan, Jonathan Protzenko, Benjamin Beurdouche, Aymeric Fromherz, Natalia Kulatova, and Santiago Zanella-Béguelin. HACLxN: Verified generic SIMD crypto (for all your favourite platforms). In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 899–918, 2020.
- [24] Jianxing Qin, Alexander Du, Danfeng Zhang, Matthew Lentz, and Danyang Zhuo. Can large language models verify system software? a case study using FSCQ as a benchmark. In *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems, HOTOS '25*, pages 34–41. Association for Computing Machinery, 2025.
- [25] Balaji Rao, John Harrison, Soonho Kong, Juneyoung Lee, and Carlo Lipizzi. s2n-bignum-bench: A practical benchmark for evaluating low-level code reasoning of llms. *arXiv preprint arXiv:2603.14628*, 2026.
- [26] Martin Riddell, Ansong Ni, and Arman Cohan. Quantifying contamination in evaluating code generation capabilities of language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14116–14137, 2024.
- [27] Chuyue Sun, Ying Sheng, Oded Padon, and Clark Barrett. Clover: Closed-loop verifiable code generation. In *International Symposium on AI Verification*, 2024.
- [28] Amitayush Thakur, Jasper Lee, George Tsoukalas, Meghana Sistla, Matthew Zhao, Stefan Zetsche, Greg Durrett, Yisong Yue, and Swarat Chaudhuri. Clever: A curated benchmark for formally verified code generation. *arXiv preprint arXiv:2505.13938*, 2025.
- [29] Kyle Thompson, Nuno Saavedra, Pedro Carrott, Kevin Fisher, Alex Sanchez-Stern, Yuriy Brun, João F Ferreira, Sorin Lerner, and Emily First. Rango: Adaptive retrieval-augmented proving for automated software verification. In *International Conference on Software Engineering (ICSE)*, 2025.
- [30] Huajian Xin, Luming Li, Xiaoran Jin, Jacques Fleuriot, and Wenda Li. Ape-bench i: Towards file-level automated proof engineering of formal math libraries. *arXiv preprint arXiv:2504.19110*, 2025.

- [31] Yutong Xin, Qiaochu Chen, Greg Durrett, and Işıl Dillig. Verisoftbench: Repository-scale formal verification benchmarks for lean. *arXiv preprint arXiv:2602.18307*, 2026.
- [32] Chenyuan Yang, Xuheng Li, Md Rakib Hossain Misu, Jianan Yao, Weidong Cui, Yeyun Gong, Chris Hawblitzel, Shuvendu Lahiri, Jacob R Lorch, Shuai Lu, et al. Autoverus: Automated proof generation for rust code. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2):3454–3482, 2025.
- [33] Chenyuan Yang, Natalie Neamtu, Chris Hawblitzel, Jacob R Lorch, and Shan Lu. Verusage: A study of agent-based verification for rust systems. *arXiv preprint arXiv:2512.18436*, 2025.
- [34] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- [35] Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Neural Information Processing Systems (NeurIPS)*, 2023.
- [36] Zhe Ye, Zhengxu Yan, Jingxuan He, Timothe Kasriel, Kaiyu Yang, and Dawn Song. Verina: Benchmarking verifiable code generation. *arXiv preprint arXiv:2505.23135*, 2025.
- [37] Lingfei Zeng, Fengdi Che, Xuhan Huang, Fei Ye, Xu Xu, Binhang Yuan, and Jie Fu. Veriequiv-bench: An equivalence score for ground-truth-free evaluation of formally verifiable code. *arXiv preprint arXiv:2510.06296*, 2025.
- [38] Haoyu Zhao, Ziran Yang, Jiawei Li, Deyuan He, Zenan Li, Chi Jin, Venugopal V Veeravalli, Aarti Gupta, and Sanjeev Arora. Algoveri: An aligned benchmark for verified code generation on classical algorithms. *arXiv preprint arXiv:2602.09464*, 2026.
- [39] Si Cheng Zhong and Xujie Si. Towards repository-level program verification with large language models. In *Proceedings of the 1st ACM SIGPLAN International Workshop on Language Models and Programming Languages*, pages 27–39, 2025.

A Experimental Configuration

A.1 Agent Harnesses and Models

Codex (v0.140.0) is invoked with GPT-5.5. The reasoning-effort setting is the Codex-side parameter `model_reasoning_effort`; the default value (medium) corresponds to the GPT-5.5 (mid) row in Section 4 and the `xhigh` value corresponds to the GPT-5.5 (xhigh) row. Claude Code (v2.1.191) is invoked with Claude Opus 4.8 and Claude Sonnet 5, both at `xhigh` reasoning effort. All agents have access to the project’s Lean toolchain in version v4.29.1 and to `lake build` commands.

A.2 Evaluation Cost

Table 2 reports the aggregate evaluation cost for each (agent, mode) cell, summed over all 43 instances.

Table 2: Aggregate evaluation cost (USD) per cell, summed over all 43 instances.

Agent	Code-and-proof	Proof-only
GPT-5.5 (xhigh)	\$2,865	\$2,964
GPT-5.5 (medium)	\$928	\$1,094
Claude Opus 4.8	\$1,983	\$2,279
Claude Sonnet 5	\$633	\$791

B Benchmark Sources and Licenses

Table 3 lists the upstream source, language, and license for each of the 43 active instances. All Track 1 instances are translated from formal-source projects (Dafny, Verus, Coq); all Track 2 instances are translated from Python. Each instance’s manifest records the upstream repository, license, and pinned commit hash. Three active instances derive from copyleft upstreams: `flocq` (LGPL-3.0-or-later), `huffman` (LGPL-2.1-or-later), and `portion` (LGPL-3.0-or-later). The `arithmetic` instance is translated from the `verus-lang/verus` example suite.

Table 3: Upstream sources, languages, licenses, and pinned commits for the 43 active instances. Commit hashes are short SHAs of the upstream snapshot used by the curation pipeline. Every instance is translated into a novel Lean 4 formalization; no Lean 4 ground truth is publicly available.

Instance	Upstream source	Lang.	License	Commit
<i>Track 1: Formal-source instances</i>				
<code>arithmetic</code>	<code>verus-lang/verus (vstd)</code>	Verus	MIT	8c06fbd7
<code>dedekind_reals</code>	<code>rocq-community/dedekind-reals</code>	Coq	MIT	da4a7452
<code>deposit_sc</code>	<code>ConsensSys/deposit-sc-dafny</code>	Dafny	Apache-2.0	cf321d10
<code>flocq</code>	<code>gitlab.inria.fr/flocq/flocq</code>	Coq	LGPL-3.0-or-later	7aab8f55
<code>huffman</code>	<code>rocq-community/huffman</code>	Coq	LGPL-2.1-or-later	cc7d4cc4
<code>json</code>	<code>dafny-lang/libraries</code>	Dafny	MIT	b486ff7f
<code>piggybank</code>	<code>AU-COBRA/ConCert</code>	Coq	MIT	341f440a
<code>sequences</code>	<code>dafny-lang/libraries</code>	Dafny	MIT	b486ff7f
<code>unicode</code>	<code>dafny-lang/libraries</code>	Dafny	MIT	b486ff7f
<code>verdict</code>	<code>secure-foundations/verdict</code>	Verus	MIT / Apache-2.0	9bc18bc5
<code>verified_bitmasks</code>	<code>achreto/verified-bitmasks</code>	Verus	MIT	cce6985c
<code>verified_ironkv</code>	<code>verus-lang/verified-ironkv</code>	Verus	MIT	08be20c3
<code>vest</code>	<code>secure-foundations/vest</code>	Verus	MIT	db63c23b
<i>Track 2: Python-source instances</i>				
<code>base58</code>	<code>keis/base58</code>	Python	MIT	2fae7065
<code>cachetools</code>	<code>tkem/cachetools</code>	Python	MIT	48284d73
<code>croniter</code>	<code>kiorky/croniter</code>	Python	MIT	9810279c
<code>diffli</code>	<code>python/cpython</code>	Python	PSF-2.0	669299b6
<code>dijkstra</code>	<code>wylee/dijkstra</code>	Python	MIT	aa1237a8
<code>ecdsa</code>	<code>tlsfuzzer/python-ecdsa</code>	Python	MIT	bff40c6c
<code>galoistools</code>	<code>sympy/sympy</code>	Python	BSD-3-Clause	2f9c274d
<code>greenery</code>	<code>qntm/greenery</code>	Python	MIT	588f5e30
<code>intervaltree</code>	<code>chaimleib/intervaltree</code>	Python	Apache-2.0	1bc406e1
<code>ipaddress</code>	<code>python/cpython</code>	Python	PSF-2.0	669299b6
<code>jsonpatch</code>	<code>stefankoegl/python-json-patch</code>	Python	BSD-3-Clause	0b052032
<code>linked_list</code>	<code>TheAlgorithms/Python</code>	Python	MIT	7a0fee40
<code>munkres</code>	<code>bmc/munkres</code>	Python	Apache-2.0	ac8af9e3
<code>netaddr</code>	<code>netaddr/netaddr</code>	Python	BSD-3-Clause	d340feab
<code>networkx</code>	<code>networkx/networkx</code>	Python	BSD-3-Clause	19509286
<code>ntheory</code>	<code>sympy/sympy</code>	Python	BSD-3-Clause	1a2501b3
<code>packaging_version</code>	<code>pypa/packaging</code>	Python	Apache-2.0 / BSD-2	dcac24cc
<code>portion</code>	<code>AlexandreDecan/portion</code>	Python	LGPL-3.0-or-later	b771acfa
<code>primefac</code>	<code>elliptic-shiho/primefac-fork</code>	Python	MIT	28adf4aa
<code>primepy</code>	<code>janaindrajit/primePy</code>	Python	MIT	9c98276f
<code>prolepticgregorian</code>	<code>python/cpython</code>	Python	PSF-2.0	669299b6
<code>pyradix</code>	<code>mjschultz/py-radix</code>	Python	ISC	b5e4e930
<code>pythonconstraint</code>	<code>python-constraint/python-constraint</code>	Python	BSD-2-Clause	f1359ff0
<code>reedsolo</code>	<code>tomerriliba-org/reedsolomon</code>	Python	Unlicense / MIT-0	796639ca
<code>rsa</code>	<code>sybrenstuvell/python-rsa</code>	Python	Apache-2.0	42b0e14f
<code>semver</code>	<code>rbarrois/python-semanticversion</code>	Python	BSD-2-Clause	2cbbec31
<code>sortedcontainers</code>	<code>grantjenks/python-sortedcontainers</code>	Python	Apache-2.0	3ac35863
<code>textdistance</code>	<code>life4/textdistance</code>	Python	MIT	d6a68d61
<code>textwrap</code>	<code>python/cpython</code>	Python	PSF-2.0	669299b6
<code>toposort</code>	<code>ericvsmith/toposort</code>	Python	Apache-2.0	9fcd0437

Curation effort. Hands-on human effort ranges from several hours for straightforward instances to a few days when translation repairs or specification revisions are required, concentrated on source selection, semantic review of translated definitions, Track 2 specification review, and build validation.

Curation consumes approximately \$60 of model usage per instance at standard API rates, including QA and validation runs in both task modes.

C Per-Instance Results

Table 4 reports per-instance pass counts at each agent’s final submitted state for every (agent, mode) cell across the 43 active instances.

Table 4: Per-instance pass counts (passed / total scored specifications) at each agent’s final submitted state, across all evaluated cells for the 43 instances. “cp” = code-and-proof, “po” = proof-only.

Instance	GPT-5.5 (xhigh)		GPT-5.5 (medium)		Claude Opus 4.8		Claude Sonnet 5	
	cp	po	cp	po	cp	po	cp	po
<i>Track 1: Formal-source instances</i>								
arithmetic	191/191	189/191	139/191	191/191	191/191	191/191	128/191	117/191
dedekind_reals	0/82	0/82	0/82	0/82	0/82	0/82	0/82	0/82
deposit_sc	79/79	79/79	63/79	65/79	71/79	74/79	21/79	48/79
flocq	155/203	154/203	138/203	150/203	153/203	166/203	119/203	148/203
huffman	127/127	92/127	14/127	19/127	0/127	116/127	42/127	72/127
json	64/64	61/64	61/64	64/64	42/64	64/64	50/64	35/64
piggybank	17/23	17/23	17/23	17/23	17/23	17/23	17/23	17/23
sequences	84/84	84/84	84/84	25/84	21/84	71/84	0/84	65/84
unicode	0/46	36/46	0/46	32/46	0/46	40/46	0/46	26/46
verdict	108/119	97/119	109/119	0/119	112/119	52/119	91/119	0/119
verified_bitmasks	124/124	124/124	105/124	105/124	103/124	124/124	71/124	69/124
verified_ironkv	19/22	22/22	17/22	14/22	19/22	19/22	11/22	14/22
vest	42/42	42/42	40/42	42/42	42/42	42/42	33/42	38/42
<i>Track 2: Python-source instances</i>								
base58	25/53	20/53	19/53	19/53	0/53	35/53	6/53	11/53
cachetools	73/74	74/74	54/74	45/74	35/74	68/74	35/74	0/74
croniter	55/55	55/55	49/55	52/55	54/55	53/55	45/55	46/55
diffliib	49/49	49/49	29/49	42/49	33/49	26/49	24/49	0/49
dijkstar	43/43	35/43	32/43	11/43	0/43	29/43	11/43	11/43
ecdsa	48/48	48/48	43/48	37/48	46/48	48/48	39/48	31/48
galoistools	30/48	14/48	16/48	14/48	17/48	20/48	15/48	20/48
greenery	5/26	2/26	5/26	2/26	5/26	2/26	5/26	2/26
intervaltree	56/56	55/56	34/56	16/56	0/56	8/56	8/56	13/56
ipaddress	68/68	68/68	49/68	68/68	44/68	48/68	21/68	42/68
jsonpatch	27/27	27/27	24/27	27/27	27/27	27/27	23/27	27/27
linked_list	109/109	109/109	108/109	87/109	109/109	109/109	109/109	81/109
munkres	19/19	12/19	19/19	12/19	19/19	12/19	7/19	12/19
netaddr	40/40	40/40	23/40	23/40	24/40	24/40	14/40	18/40
networkx	40/40	40/40	24/40	24/40	39/40	22/40	20/40	20/40
ntheory	43/62	48/62	34/62	36/62	0/62	42/62	30/62	29/62
packaging_version	70/74	74/74	69/74	31/74	22/74	51/74	26/74	0/74
portion	63/63	63/63	13/63	29/63	0/63	0/63	20/63	9/63
primefac	56/56	56/56	20/56	51/56	43/56	47/56	16/56	18/56
primepy	9/9	9/9	8/9	6/9	9/9	9/9	8/9	7/9
prolepticgregorian	61/61	52/61	58/61	0/61	32/61	38/61	15/61	18/61
pyradix	52/52	52/52	50/52	52/52	35/52	1/52	45/52	31/52
pythonconstraint	20/20	20/20	18/20	18/20	20/20	20/20	20/20	20/20
reedsolo	0/48	27/48	12/48	9/48	14/48	19/48	7/48	9/48
rsa	59/63	63/63	30/63	33/63	49/63	33/63	35/63	29/63
semver	52/53	53/53	52/53	52/53	48/53	53/53	35/53	22/53
sortedcontainers	49/49	24/49	15/49	18/49	33/49	40/49	14/49	11/49
textdistance	59/59	59/59	20/59	47/59	17/59	58/59	0/59	25/59
textwrap	60/60	60/60	39/60	39/60	50/60	42/60	29/60	20/60
toposort	12/15	15/15	6/15	5/15	15/15	10/15	5/15	6/15

D Anti-Cheating Mechanisms

The grader must decide whether an agent has verified a repository or merely arranged for the checker to agree with it. Vero enforces this through three independent layers, described in turn below. Each layer is deterministic except the final judge, and each rejects submissions that a naive “does it compile” check would accept.

Layer 1: slot-scoped re-rendering. An agent works in a sandbox containing the full repository, but only the interiors of `!benchmark` marker pairs are treated as its submission. The grader extracts those interiors and overlays them onto a fresh copy of the pristine instance before compiling anything. Every declaration outside a marker pair therefore reverts to the curated original, which removes an entire family of attacks at once: weakening a specification, redefining a frozen helper, relaxing a type signature, or deleting a proof obligation.

Layer 2: axiom allowlist. Compilation alone does not establish that a proof is honest, because Lean admits `sorry` and user-declared axioms as well-typed terms. After the build succeeds, the grader emits `#print axioms` for every scored theorem and classifies its dependency set. Only the three standard axioms of Lean’s logic are accepted, namely `Classical.choice`, `propext`, and `Quot.sound`. A curator may extend this set per instance through `manifest.trusted_axioms`, which is recorded in the released benchmark. A theorem that depends on `sorryAx` grades as unfilled, and one that depends on any other axiom grades as `axiom_leaked`. Neither counts as passed.

This layer is not hypothetical. Across the corpus 368 specification outcomes are rejected at this stage, and they concentrate in instances with large finite domains: `base58` (67), `verified_bitmasks` (60), `reedsolo` (53), and `sortedcontainers` (52). The dominant pattern is `native_decide`, which discharges a decidable goal by evaluating it in the compiler rather than in the kernel and records a dependency on `Lean.ofReduceBool`. On `base58`, for instance, GPT-5.5 (xhigh) closes the `Base58` alphabet round-trip this way:

```
1 theorem b58Idx_b58Char_fin (d : Fin 58) :  
2   b58Idx (b58Char d.val) = some d.val := by  
3   native_decide +revert
```

The claim is true and the tactic is legitimate Lean, but it is not a kernel-checked proof and it does not generalize beyond the finite domain it enumerates. Twenty-eight of that cell’s 53 specifications fail for this reason while 25 pass, which is why the failure-reason breakdown in Figure 7(b) separates rejected shortcuts from unattempted obligations.

Layer 3: declaration screening. The final layer inspects agent-introduced declarations for constructions that decouple logical semantics from executable behavior. For `typeclass` instances, a rule-based prefilter raises two signals, a hollow constant body and a `priority` override that shadows a standard-library instance, and an LLM judge then classifies each flagged site as `legit` or `cheat` from a window of surrounding source. When both signals fire together the site is classified as a cheat without consulting the judge.

A separate deterministic check covers the case where an agent splits the proof target from the runtime function, described last below.

Each pattern is invisible to the first two layers, because each compiles and none introduces an axiom. We give one example of each.

Hollow body on a non-trivial typeclass. An ordering instance whose body ignores both arguments makes every comparison reduce to `True`:

```
1 instance : LE Float := <fun _ _ => True>
```

Any specification phrased over `<=` then closes by `decide` regardless of what the implementation computes. The signal is that the standard meaning of `LE` on `Float` is not constant, so a body that discards its arguments cannot be computing it.

Priority shadowing. The same hollow body becomes more dangerous when it displaces a real instance rather than filling a gap:

```
1 instance (priority := 2000) : LT Float := <fun _ _ => True>
```

Lean resolves standard-library instances at priority around 1000, so a declared priority of 2000 takes precedence over `Float`’s genuine ordering everywhere in the repository, including inside specifications the agent never touched. Because this pattern combines two independent signals, the prefilter classifies a hollow body together with a priority override as a cheat without consulting the judge.

Decidability laundering. A Decidable instance may assert its own verdict instead of computing one:

```
1 instance : DecidableEq (Foo -> Bar) := fun _ _ => .isTrue (by sorry)
2 theorem prove_x : spec_x canonical := by decide
```

Here the axiom check does catch the `sorry`, but the variant `.isTrue (by trivial)` on a proposition that happens to be provable in a trivial instance, or `:= trivial` supplied directly as a Decidable body, leaves no axiom trace while still making `decide` report `isTrue`.

Splitting the proof target from the runtime function. The most consequential pattern does not involve typeclasses at all, so we screen it separately. An agent can define the scored API as a noncomputable copy of the specification’s own witness oracle, and then attach a real algorithm through `@[implemented_by]`:

```
1 noncomputable def findPathCost : FindPathCostSig :=
2   fun g s t => if h : exists c, isMinWalkCost g s t c then h.choose else 0
3   @[implemented_by realBellmanFord] def findPathCost' := findPathCost
```

Every optimality proof about `findPathCost` is then definitionally true, because the function is the specification’s answer set, while the compiled binary still runs a correct shortest-path algorithm. We encountered this during benchmark development on `dijkstra`, where the submission produced no output difference from the reference across 20,440 generated graphs and would have passed any differential test. It is also general: the technique defeats any unique-optimum specification suite no matter how the specifications are strengthened, which is why the defense has to live in the grader. The grader now rejects `@[implemented_by]` in any agent-editable `Impl/` file outright, voiding the run and grading every specification `impl_oracle`, since the attribute has no legitimate use in an implementation slot. Reference definitions on the proof side that are noncomputable without the attribute, such as `flocq`’s use of `Classical.choose`, are unaffected.

E Additional Evaluation Results

E.1 Instances Have Shared Walls, Not Random Residuals

If the specifications an agent fails were determined mostly by how much budget it happened to spend, different configurations would fail on different specifications. They do not. On several instances every configuration that gets close fails on exactly the same set.

`verdict` is the clearest example and also the corpus’s nearest miss: Claude Opus 4.8 in code-and-proof reaches 112 of 119 specifications. Its seven-specification residual is failed by all eight configurations, so it is exactly the intersection of every configuration’s failures. Four of the seven concern wildcard certificate-name matching (`spec_match_name_wildcard_base`, `_single_label`, `_multi_label_rejected`, and `spec_permit_name_dot_prefix`) and three concern projecting parsed certificate fields (`spec_cert_from_parsed_sig_alg_inner`, `_outer`, and `_validity`). An instance that no agent completes is thus 94.1% solved by the best one, and the part that resists is a wall two groups wide rather than a scattering of hard cases. Configurations differ only in how much additional ground they fail to cover: 10 residuals for GPT-5.5 (mid) in code-and-proof, 11 for GPT-5.5 (xhigh), and 28 for Claude Sonnet 5, each a superset of the same seven.

`piggybank` shows the same structure with no exceptions at all: all eight configurations pass 17 of 23 specifications and all eight fail the same six. Here the boundary is easy to characterise. The 17 that pass state properties of a single contract call, such as that inserting a negative amount fails or that smashing by a non-owner fails, and they reduce to case analysis on one invocation. The six that fail quantify over reachable chain states or over an execution trace:

```

1  def spec_balance_on_chain (impl : RepoImpl) : Prop :=
2    forall (bstate : ChainState) (caddr : Address),
3      reachable bstate ->
4        env_contracts bstate caddr = some (toWeakContract (contract impl)) ->
5          outgoing_acts bstate caddr = [] ->
6            exists cstate : State,
7              contract_state bstate caddr = some cstate /\
8                env_account_balances bstate caddr = cstate.balance

```

Discharging this requires induction over the reachability relation with a contract invariant that no single call establishes, which is the inductive-generalization gap of Appendix H appearing as a hard boundary rather than a budget limit. The division is exact: all six failing specifications quantify over a `ChainState` or a `ChainTrace`, and not one of the 17 passing specifications does. A single syntactic feature of the statement therefore predicts the outcome for every specification in the instance and for every configuration.

unicode localises the same phenomenon onto a semantic type. Claude Opus 4.8 in proof-only passes 40 of 46, and five of its six failures are encode and decode round-trips (`spec_utf16_roundtrip`, `spec_utf16_decode_encode`, `spec_uniChar_utf16_roundtrip`, `spec_noChar_utf8_roundtrip`, and `spec_noChar_utf8_decode_encode`) while six other round-trip specifications in the same instance pass, among them `spec_utf8_roundtrip` and `spec_uniChar_utf8_roundtrip`. The split follows the level of abstraction rather than the round-trip property itself. All three round-trips stated at the encoding-form level pass, including `spec_utf16_encode_decode_roundtrip`, so UTF-16 is not the obstacle. Every failure is at the string level, where all three UTF-16 round-trips fail while the UTF-8 ones pass except in the variant restricted to strings without unicode characters. The statements are near-identical in shape, each asserting that a checked encode followed by a checked decode recovers the input, so the difficulty sits in the supplied string-level conversion functions the proofs must unfold rather than in what the specifications say.

E.2 Implementation Freedom Helps Only the Strongest Agent

The main text reports that code-and-proof and proof-only each win instances the other loses. Disaggregating by model shows the effect is not symmetric across agents. Counting instances where exactly one mode is a full solve, GPT-5.5 (xhigh) is code-and-proof-only on 8 and proof-only-only on 6. Every weaker configuration leans the other way, with GPT-5.5 (mid) at 2 against 6, Claude Opus 4.8 at 2 against 4, and Claude Sonnet 5 at 1 against 1. At specification granularity, code-and-proof gains GPT-5.5 (xhigh) 42 specifications and GPT-5.5 (mid) 130, but costs Claude Opus 4.8 360.

The cost is not proof difficulty. Of Opus’s 1,095 residual specifications in code-and-proof, 1,074 are unattempted and only 21 are rejected for axiom use, and none fails to build. In proof-only its residual falls to 735 and is entirely unattempted. The trajectory data explain why. Claude Opus 4.8 authors a median of 6 implementation and 177 proof lines by minute 2 of a code-and-proof run and is still at 195 proof lines by minute 30, reaching 811 only in the final half hour. GPT-5.5 (xhigh) already has 45 implementation and 833 proof lines at minute 2 and adds only 474 more over the remaining 88 minutes. Opus therefore spends most of its wall clock before its proof text exists, so an added implementation obligation delays the part of the run that scores. Six instances where its code-and-proof score collapses to zero (Huffman, ntheory, unicode, base58, dijkstar, and intervaltree) score 8 to 116 in proof-only, and in every one of its eight zero cells the grader finds no filled route at all rather than a failed one.

In short, code-and-proof measures two capabilities at once, and for agents that are slow to produce a first artifact the implementation obligation dominates. This is worth separating from the substitution effect of Appendix G, which requires enough budget to first write an implementation and then exploit it.

E.3 Weaker Agents Add Nothing to the Frontier

A benchmark can be nearly saturated by an ensemble even when no single agent saturates it, so we ask how much the four agents cover together.

At repository granularity the answer is that they cover no more than the strongest one. GPT-5.5 (xhigh) fully solves 33 distinct instances across its two modes. The union over all eight configurations is also 33: not one instance is completed by a weaker configuration that GPT-5.5 (xhigh) misses in both modes. The three weaker agents together solve 15 instances, every one of which is a subset of what GPT-5.5 (xhigh) already solves. Ensembling therefore buys nothing at the repository level, and the ten instances that no configuration completes resist the entire evaluation rather than resisting individual agents.

At specification granularity there is modest headroom. Taking the union of every specification passed by any configuration gives 2,486 of 2,705 (91.9%), against 2,362 (87.3%) for the best single cell. The extra 124 specifications are spread across instances where different agents happen to close different obligations, and they leave 219 specifications that no configuration ever passes. Those 219 concentrate sharply, and every one of them falls in the ten instances that no configuration completes: 82 in `dedekind_reals`, where nothing is ever proved, then 34 in `flocq`, 21 each in `greenery` and `reedsolo`, 18 in `galoistools`, 13 in `nththeory`, 11 in `base58`, 7 in `verdict`, and 6 each in `unicode` and `piggybank`. This set is the part of Vero that is currently out of reach for every agent we tested, and it is small enough to inspect by hand, which is how we identified the reachability and abstraction-level patterns of Section E.1. We further note that a repository where its implementation only satisfies a partial set of specifications is not considered verified. Therefore, the union coverage at the specification level does not mean these specifications are saturated.

E.4 Cost of Repository-Level Verification

Table 2 reports what each configuration spent in total. Normalizing by what that spending bought gives a different ranking, shown in Table 5. These figures need a provenance caveat. Costs are measured from the run’s own telemetry for 121 of the 344 cells. The other 223 are runs that hit the wall-clock cap and were killed mid-turn before emitting a terminal usage record, so they are estimated as the model’s median dollars per minute over its measured runs times that cell’s elapsed time. The estimated share is similar for every configuration, between 53% and 66% of total spend, so the comparison across configurations is at least uniform in method, but the absolute totals are soft and we read only the large gaps below.

Table 5: Cost per unit of progress. Total spend is summed over the 43 instances, and the last two columns divide it by full solves and by passed specifications respectively.

Agent	Mode	Total	Full	\$ / full solve	Specs	\$ / spec
GPT-5.5 (xhigh)	code + proof	\$2,865	27	\$106	2,362	\$1.21
GPT-5.5 (xhigh)	proof only	\$2,964	25	\$119	2,320	\$1.28
GPT-5.5 (medium)	code + proof	\$928	2	\$464	1,759	\$0.53
GPT-5.5 (medium)	proof only	\$1,094	6	\$182	1,629	\$0.67
Claude Opus 4.8	code + proof	\$1,983	8	\$248	1,610	\$1.23
Claude Opus 4.8	proof only	\$2,279	10	\$228	1,970	\$1.16
Claude Sonnet 5	code + proof	\$633	2	\$317	1,270	\$0.50
Claude Sonnet 5	proof only	\$791	2	\$396	1,237	\$0.64

The most capable configuration is also the cheapest per completed repository. GPT-5.5 (xhigh) spends the most in absolute terms and the least per full solve, at \$106 in code-and-proof against \$182 to \$464 for every other configuration. The ordering reverses per specification, where the two weakest configurations are the cheapest at \$0.50 and \$0.53, because partial coverage accumulates on easy obligations at low cost. Which metric to prefer depends on what a practitioner wants. Paying for specifications passed favors cheap agents, and paying for repositories actually completed favors the expensive one by a factor of two to four.

Raising reasoning effort is superlinear in yield. Moving GPT-5.5 from default to xhigh effort multiplies spend by 3.1 in code-and-proof and 2.7 in proof-only, and multiplies full solves by 13.5

and 4.2. The gain is therefore larger than the price for this task, which follows from the all-or-nothing metric rather than from any superlinear gain in proof ability. Per specification the same effort increase buys far less, moving coverage from 65% to 87% in code-and-proof for triple the spend. A configuration that closes most of a repository and stops earns nothing on the full-solve metric, so effort spent near the completion boundary converts into repository counts at a rate the specification counts do not show.

Unfinished runs cost more than finished ones. For three of the four agents the median unfinished cell is more expensive than the median full solve, at \$53 against \$43 for GPT-5.5 (xhigh), \$53 against \$42 for Claude Opus 4.8, and \$15 against \$9 for Claude Sonnet 5. GPT-5.5 (mid) is the exception at \$16 against \$25, and the Sonnet and GPT-5.5 (mid) comparisons rest on four and eight full solves respectively, so we read the direction rather than the magnitudes. Agents that finish tend to finish early, and agents that do not finish spend the remaining budget on obligations they never close. Aggregated over the grid, 23% of all spending went to the ten instances that no configuration solves. This is the cost signature of the full-solve metric, and it is worth stating for anyone budgeting a run: much of the money is spent on the repositories that return nothing.

F Audit-Mechanism Case Studies

Each clean closure of an audit theorem constitutes formal evidence that a specification, as introduced in Section 3.5, does not hold on the canonical implementation (`disprove`), is universally unsatisfiable (`unsat`), or jointly contradicts a sibling specification (`joint_unsat`). We ran the mechanism throughout curation, adjudicating every closure by hand against the pinned upstream source. Restricted to the 43 released instances, it produced 38 adjudicated specification defects across 9 instances, plus 6 joint-unsatisfiability groups. All of them are repaired in the released benchmark, and the evaluation reported in Section 4 runs against the repaired version. This appendix reports the aggregate findings first and then three case studies, one for each closure route: a `joint_unsat` pair on `verdict`, an `unsat` family on `verified_ironkv`, and the largest `disprove` cluster, on `verified_bitmasks`.

A closure is only evidence if it is machine-checked, so we record the basis for each of the 38 cases separately: 34 rest on a clean certificate the agent submitted, 3 on a certificate we reconstructed after the agent’s proof failed the axiom check, and 1 on an independently reproduced native evaluation. Cases that share a root cause are not statistically independent, and we note the largest such cluster below rather than treating all 38 as separate discoveries. We also hold a further set of translated instances that we audited but did not bring to release quality, and their findings are excluded from every count here. Those instances are the main reason the released corpus is 43 rather than larger.

Table 6: Audit-mechanism findings during benchmark development. “Specs” is the number of distinct specifications flagged across all configurations.

Defect category	Cases	Share	Instances	Largest contributor
Mistranslated definition or semantics	18	47.4%	3	<code>huffman</code> (16)
Missing domain or operation condition	17	44.7%	6	<code>verified_bitmasks</code> (7)
Reference implementation mismatch	2	5.3%	1	<code>flocq</code> (2)
Direct conflict between specifications	1	2.6%	1	<code>verdict</code> (1)
Total	38		9	

A second auditor adds little. We ran the audit with two agents, and their findings overlap heavily. Claude Opus 4.8 found 33 of the 38 cases and Codex found 30, with 25 found by both. Adding the weaker finder to the stronger one therefore contributes 5 additional cases, a 15% increment over the stronger single auditor. This is useful for calibration: the marginal value of another auditor is real but modest, and the cases both agents find independently are the ones we adjudicated fastest, since two unrelated counterexamples for one specification leave little doubt about the verdict.

Audit 1: two specifications in direct conflict on `verdict` (`joint_unsat`, `fixed`). Two specifications govern the Base64 character classifier `charToBits`:

```

1 def spec_char_to_bits_padding (impl : RepoImpl) : Prop :=
2   impl.verdict.charToBits 0x3D = some none
3
4 def spec_char_to_bits_reject (impl : RepoImpl) : Prop :=
5   ∀ (c : UInt8),
6     (c < 0x2B ∨ (c > 0x2B ∧ c < 0x2F) ∨
7      (c > 0x39 ∧ c < 0x41) ∨ (c > 0x5A ∧ c < 0x61) ∨ c > 0x7A) →
8     impl.verdict.charToBits c = none

```

The padding character ‘=’ (0x3D) lies in the gap (0x39,0x41) between ‘9’ and ‘A’, so `spec_char_to_bits_reject` requires `charToBits 0x3D = none` while `spec_char_to_bits_padding` requires it to return `some none`. No implementation can satisfy both. GPT-5.5 (xhigh) flagged this pair through a joint-unsatisfiability proof that collapses to a one-line $0x3D < 0x41 \wedge 0x3D > 0x39$ contradiction after unfolding both specifications. The fix excludes 0x3D from the reject range by adding a $c \neq 0x3D$ conjunct:

```

1   (c > 0x39 ∧ c < 0x41 ∧ c ≠ 0x3D) ∨   -- excluded '='

```

This error survived manual review and type-checking but was caught automatically by the agent.

Audit 2: comparator-lawfulness gap on `verified_ironkv` (unsat, fixed). Three specifications on `verified_ironkv` quantify over an arbitrary `KeyTrait K` without constraining comparator lawfulness. GPT-5.5 (xhigh) closed all three via unsat proofs by instantiating an adversarial comparator. For example, the witness for `spec_strictly_ordered_vec_no_duplicates` uses a constant comparator:

```

1  theorem unsat_strictly_ordered_vec_no_duplicates :
2    ¬ (∃ impl : RepoImpl,
3      spec_strictly_ordered_vec_no_duplicates impl) := by
4    intro ⟨impl, h⟩
5    -- Adversarial KeyTrait: cmpSpec returns Less for every pair.
6    let bad : KeyTrait Unit :=
7      { zeroSpec := (), cmpSpec := fun _ _ => Ordering.Less }
8    -- A two-element vector of () is valid under bad but has duplicates.
9    let s : StrictlyOrderedVec Unit := ⟨[()], ()⟩
10   exact (h _ bad s (by intro i j _; rfl)) ⟨0, 1⟩ rfl

```

We resolved this by introducing a `LawfulKeyTrait` class capturing the minimal comparator laws (equality iff `.Equal`) and threading it through the three affected specifications.

Audit 3: missing domain conditions on `verified_bitmasks` (disprove, fixed). The largest cluster of disprove closures in a released instance is 7 specifications on `verified_bitmasks`, and it separates into two root causes that both amount to a missing domain condition.

Three specifications assert that a bitwise operation preserves the bit count of its first argument, but they quantify over bitmasks of unequal length. Because the implementation combines the arguments with `zipWith`, the result has length $\min |A| |B|$, so any pair with $|B| < |A|$ refutes the claim. The repair adds the length precondition that the sibling `bSeq` specifications already carried:

```

1  def spec_bIIF_and_nbits_preserved (impl : RepoImpl) : Prop :=
2    forall (A B : T), A.length = B.length -> -- added
3      impl.verifiedBitmasks.bIIF_nbits (impl.verifiedBitmasks.bIIF_and A B) =
4      impl.verifiedBitmasks.bIIF_nbits A

```

The other four specifications compare a population count against a machine-word bound, and they fail once a bitmask is long enough for the count to exceed what a 64-bit word represents. This case is also the clearest example of why every closure passes through human adjudication. Our first repair draft compared the counts in `Nat` instead of in the machine type, which is itself unsound because the residues still invert at lengths beyond 2^{64} . The second auditor rejected that draft and we adopted a uniform representability precondition for all four specifications instead.

After repair we did not merely re-run the audit to confirm the specifications are no longer refutable. We wrote and compiled a positive certificate `theorem _chk_X : spec_X canonical` for each of the 7, so the record shows they are genuinely true of the reference implementation rather than only beyond the reach of the previous counterexample.

G Implementation Freedom in code-and-proof Mode

code-and-proof mode lets an agent choose the implementation it will prove against, and the strongest agents exploit this by replacing a reference algorithm that is hard to reason about with one that is easy to reason about. We audited the source of every code-and-proof full solve whose proof-only counterpart is not a full solve and identified at least five instance-agent pairs where the submitted implementation is algorithmically different from the reference rather than a stylistic rewrite. Across those five, the agent closes all 250 specifications in code-and-proof, while proving the identical specifications against the fixed reference in proof-only closes only 201.

Replacing an algorithm with a simpler one. `munkres` is the clearest case, and all three of GPT-5.5 (xhigh), GPT-5.5 (mid), and Claude Opus 4.8 make the same move. The reference implementation is a faithful port of the Hungarian algorithm: a 409-line six-state machine driven by a fuel-bounded `runSteps` loop with 38 auxiliary definitions covering starring, priming, and covering-line book-keeping. Reasoning about its output requires an invariant relating the internal state to assignment optimality at every step. Instead, each agent submits a 164-line implementation that enumerates permutations and selects a minimum-cost one:

```
1 def permsAuxImpl : List Nat -> List (List Nat)
2   | [] => [[]]
3   | x :: xs => (permsAuxImpl xs).flatMap (fun p => insertEverywhereImpl x p)
4
5 def permCostImpl (M : List (List Nat)) (p : List Nat) : Nat :=
6   sumN ((List.range p.length).map (fun i => matGetN M i (natGet p i)))
7
8 def bestPermFrom (M : List (List Nat)) (best : List Nat) : List (List Nat) -> List Nat
9   | [] => best
10  | p :: ps =>
11    if permCostImpl M p < permCostImpl M best then bestPermFrom M p ps
12    else bestPermFrom M best ps
```

This implementation is exponential and would be unusable in production, but it is close to a transcription of what the specifications assert, so optimality follows by induction over the candidate list rather than from a loop invariant. The mode gap is entirely in those obligations: in proof-only every configuration closes exactly 12 of 19 specifications, and the seven failures are precisely the optimality and global-bound claims (`spec_compute_optimal_value`, `spec_compute_achieves_min`, `spec_compute_feasible`, `spec_compute_profit_max`, `spec_compute_cost_colVec`, `spec_compute_colVec_mem`, and `spec_compute_cost_global_bounds`). The structural specifications, which do not depend on the search strategy, pass in both modes.

Delegating to the standard library. The other two audited pairs substitute a library routine for a hand-written one. On sequences, GPT-5.5 (xhigh) implements the merge-sort API as `fun a lessThanOrEq _h => a.mergeSort lessThanOrEq`, and on `linked_list` Claude Sonnet 5 implements list merging as `fun l => l.mergeSort (· ≤ ·)`. Both replace a recursive definition the curator wrote with Lean’s `List.mergeSort`, whose sortedness and permutation lemmas already exist in the standard library. The effect is visible in the `linked_list` pair, where Sonnet 5 closes 109 of 109 specifications in code-and-proof but 81 in proof-only: the same proofs are available in both modes, but only in code-and-proof can the agent choose the definition those proofs are stated about.

When implementation freedom hurts. The converse effect is larger across the corpus. Seventeen instance-agent pairs are full solves in proof-only but not in code-and-proof, and the residuals there are dominated by unattempted obligations rather than rejected proofs. Huffman with Claude Opus 4.8 is one of the example: 116 of 127 specifications in proof-only versus 0 in code-and-proof. The cause is not proof difficulty but the re-render boundary of Appendix D. The agent left three implementation slots in `Impl/Build.lean` empty, so the graded artifact instantiates them with `sorry` where a function is required, and the build fails:

```
1 error: Huffman/Impl/Build.lean:69:32: Function expected at
2   ObuildfSig
3   but this term has type
```

Because Vero grades the repository rather than the specification, a single unbuildable implementation module voids every specification in the instance: the grader marks the submission `impl_broken` and awards no partial credit. Seven of the 344 cells end this way, and all seven score zero. This is the mechanism behind the mode asymmetry in Figure 4(a): proof-only hands the agent a repository that already compiles, whereas code-and-proof requires it to keep the repository compiling while it works.

H Hard-Case Proof Studies

The figures in Section 4 report what agents achieve. This appendix reports how they work, by reading the artifacts and the files agents leave behind in the sandbox. We cover four instances: `deposit_sc` (the most substantive proof-engineering effort we observed), `vest` (where case analysis migrates out of the obligations), `sortedcontainers` (where the two task modes diverge sharply), and `galoistools` (where an agent’s search strategy is visible in its scratch files).

H.1 Agents Leave Their Search Visible in the Sandbox

Agents work in a sandbox containing the whole repository, and only marker interiors are graded. Some agents exploit this freedom to build a private workspace: they create Lean files outside the graded regions, use them to probe the standard library and draft proofs, and leave them behind at the end of the run. These files are not scored, but they record the agent’s search in a way no summary statistic does.

The habit divides the agents sharply. Claude Opus 4.8 creates such files in 32 of its 86 cells and Claude Sonnet 5 in 31, together writing 360 of them. GPT-5.5 (xhigh) does so in 2 cells and GPT-5.5 (mid) in 10. This is not a proxy for skill, and the direction is the opposite of what one might expect: for Opus, cells with a scratch workspace pass 59.0% of specifications against 69.8% for cells without one, and produce 2 full solves against 16. Scratch files appear where an agent is stuck, so they are best read as a record of what stuck looks like.

Probing before proving. The first thing an agent does with a private file is ask what already exists. Claude Sonnet 5 on arithmetic opens with a file that contains no proofs at all, only queries against the standard library:

```
1  import ArithmeticV2.Harness
2  import ArithmeticV2.Spec.DivMod
3
4  -- Test what lemmas exist
5  #check @Int.ediv_self
6  #check @Int.emod_lt_of_pos
7  #check @Int.ediv_add_emod
8  #check @Int.emod_emod_of_dvd
9  ...
```

Later files in the same cell shift from probing to drafting, stating a private helper and a candidate obligation proof together so that both can be checked in one compilation before either is committed to a graded slot. The agent has effectively built itself a read-eval-print loop out of files, and it is doing what a human would: establishing the available vocabulary first, then composing within it. This behavior is invisible in the graded artifact, because the scratch files are discarded at grading time and the committed proofs show only the end state.

Retrying one lemma across many files. The same mechanism exposes a failure mode. On `galoistools`, Claude Opus 4.8 in code-and-proof leaves 76 scratch files containing attempts at 42 distinct lemmas, and the distribution is heavily skewed: 17 lemmas are attempted in more than three files, and one, `refGfStrip_eq_gfStripAux`, reappears in 16 of the 76. Three more (`refPolyEvalRevAux_append_zero`, `eval_mod_lemma`, and `cancel_mod`) appear in 12 or 13 files each. The cell closes 17 of 48 specifications.

What the agent is repeatedly trying to prove is a bridging lemma between the curator’s reference definition and its own implementation:

```
1  -- Are refGfStrip and gfStripAux the same?
2  theorem refGfStrip_eq_gfStripAux : forall (f : List Nat),
3    refGfStrip f = gfStripAux f := by
4    intro f
5    induction f with
6    | nil => rfl
7    | cons a as ih => simp [refGfStrip, gfStripAux]; split; exact ih; rfl
```

The agent has diagnosed the problem correctly. Its own `gfStripAux` and the reference `refGfStrip` are extensionally equal, and the obligations need that fact, so the lemma is the right thing to want. Across the sixteen files the statement stays fixed while the tactic script varies, and it also keeps returning to the lemma after intervening work on other goals rather than pursuing it to a conclusion in one sitting.

The alternative the agent never takes is to remove the need for the lemma. Because this is code-and-proof mode, it controls `gfStripAux` and could define it to be syntactically the reference function, at which point the agreement lemma becomes `rfl`. This is precisely the substitution move that succeeds on `munkres` and `sortedcontainers` (Appendix G and section H.4), applied in reverse: instead of choosing an implementation that is easy to reason about, the agent commits to one that differs from the reference and then pays for the difference in every proof.

Implication for agent design. Two properties of this behavior suggest concrete changes. First, the probing phase is cheap and effective, and the agents that skip it are the ones that already know the library. An agent with weaker priors benefits from being asked to enumerate the available lemmas before drafting. Second, the retry loop has no step that revisits the implementation. The agent treats its own definitions as fixed once written, so a mismatch it introduced in the first minutes becomes a proof obligation it pays for over the remaining ninety. This is the same one-way ordering that Figure 6 shows across all configurations, where implementation size plateaus early while proof text grows to the deadline. A loop that, on repeated failure of a bridging lemma, reconsidered the definition rather than the tactic would turn sixteen attempts at one lemma into one edit that discharges it.

H.2 `deposit_sc`: A Layered Proof Library

`deposit_sc` is a Lean port of the ConsenSys Dafny Merkle-tree incremental-commitment contract; the instance contains 79 specifications across four modules (`Bits`, `Tree`, `Merkle`, `Contract`). GPT-5.5 (xhigh) closes all 79 in both task modes. The code-and-proof submission is the largest proof artifact in the corpus: 6,052 nonblank lines across the four proof modules, containing 99 agent-written helper theorems alongside the 79 specification solutions. The library is layered to match the module hierarchy, with bit-list arithmetic (`Bits`, 846 lines), structural facts about complete decorated trees (`Tree`, 1,384), `buildMerkle` and root-computation correctness (`Merkle`, 3,479), and state-transformer frame conditions (`Contract`, 343), so that each tier depends only on the ones below it.

The work happens in named helpers. 54.3% of the proof text sits in helper theorems rather than in the obligations themselves, and the two populations have very different shapes: the median obligation proof is 9 lines while the median helper is 19, and the longest helper runs 262 lines. The effect is that most obligations reduce to a single application. `prove_same_leaves_same_root` states that two complete decorated trees with the same height and leaf sequence have equal roots, and its proof is a two-line delegation to a 94-line induction the agent proved separately:

```
1 theorem prove_same_leaves_same_root : spec_same_leaves_same_root canonical := by
2   intro t t' f hc hc' hd hd' hh hleaves
3   exact tree_same_leaves_same_root_aux t t' f hc hc' hd hd' hh hleaves
```

This is the division of labor the aggregate statistics in Figure 5 describe, visible in one instance: the agent decides what the reusable fact is, proves it once under a name, and then discharges specifications by citing it.

Strengthening the induction hypothesis. The interesting proofs are the ones where the obligation as stated is not directly inductive. `spec_incremental_equals_batch` asserts that folding a batch of deposits agrees with depositing them one at a time. Proving it by induction on the batch fails, because the inductive step needs to know that the intermediate state is still valid and that its height, merge function, and default are unchanged. The agent’s proof introduces a conjunctive auxiliary claim carrying all five facts at once and induct on that:

Choosing that conjunction is the step our depth analysis identifies as the discriminator between configurations: the fact needed at depth four is not a harder version of the fact needed at depth zero, it is a differently stated one.

```

1  have aux : forall (s : DepositState) (vs : List Int),
2    Valid canonical s →
3    s.values.length + vs.length < canonical.depositSc.power2 s.treeHeight →
4    Valid canonical (vs.foldl DepositSc.deposit s) ∧
5    (vs.foldl DepositSc.deposit s).values = s.values ++ vs ∧
6    (vs.foldl DepositSc.deposit s).treeHeight = s.treeHeight ∧
7    (vs.foldl DepositSc.deposit s).f = s.f ∧
8    (vs.foldl DepositSc.deposit s).default = s.default := by
9    intro s vs
10   induction vs generalizing s with
11     ...

```

What changed relative to earlier evaluations. An earlier round of this evaluation left 7 of the 79 specifications unproven, all depending on one missing boundary-shift lemma about how Merkle-path siblings evolve when the leaf array grows by one element. Those seven included `spec_deposit_preserves_valid`, the contract’s principal inductive invariant. In the current evaluation the agent closes that invariant, and its proof is where the remaining difficulty concentrated: a 36-line argument that deconstructs the eight-way validity conjunction, rebuilds the sibling relation for the extended leaf array, and re-establishes each component. The longest single proof in the submission is now `prove_compute_new_left_path_correct` at 524 lines. The limitation we described previously was therefore a budget and organization boundary rather than a capability ceiling: given the same specifications, the same configuration eventually found the generalization it had missed.

H.3 vest: Parser Branch Reasoning

`vest` is a Lean port of selected modules from the `secure-foundations/vest` verified-parser project, with 42 specifications spread thinly across 13 modules, none of which carries more than eight. GPT-5.5 (xhigh) and Claude Opus 4.8 close all 42 in both modes, GPT-5.5 (mid) closes 40 and 42, and Claude Sonnet 5 closes 33 and 38. The instance is therefore useful for a different question than `deposit_sc`: not whether an agent can sustain a long proof, but where the work goes when the specifications are shallow and numerous.

Case analysis migrates into helpers. The obligations themselves are short. `spec_btcvartint_parse_correct`, which relates the executable Bitcoin variable-length-integer parser to its specification, closes in six tactic lines:

```

1  theorem prove_btcvartint_parse_correct : spec_btcvartint_parse_correct canonical := by
2    intro s
3    constructor
4    . intro n v h
5      simp [canonical, VestV2.btcVarintParse, h]
6    . intro h
7      exact <ParseError.UnexpectedEndOfInput,
8        by simp [canonical, VestV2.btcVarintParse, h]>

```

The four-way case analysis the format actually requires has not disappeared. It has moved into a named helper. The submission’s longest proof is an 87-line bound on the parser’s consumed length, which splits on the leading discriminant byte to separate the single-byte, `0xFD`, `0xFE`, and `0xFF` encodings and then deconstructs enough of the payload in each branch to establish the bound:

Across the instance the agent writes 27 helper theorems for 1,893 lines of proof, roughly one helper per one-and-a-half specifications. The contrast with `deposit_sc`, where 99 helpers support 79 specifications over 6,052 lines, is the difference between a repository whose difficulty is depth and one whose difficulty is breadth: `vest` needs many small facts about independent formats, and the agent’s library is correspondingly flat.

H.4 sortedcontainers: Choosing an Algorithm You Can Induct On

For `sortedcontainers`, GPT-5.5 (xhigh) closes all 49 specifications in code-and-proof and 24 in proof-only, with the 25 residuals distributed across all three data structures (`SortedDict` 11,

```

1  theorem btc_parse_length_bounds_aux :
2    forall (s : List UInt8) (n : Int) (v : VarInt),
3      VarInt.spec_parse s = some (n, v) ->
4        1 <= n /\ n <= 9 /\ n.toNat <= s.length := by
5  intro s n v h
6  unfold VarInt.spec_parse at h
7  cases s with
8  | nil => simp at h
9  | cons b rest =>
10     by_cases hb : b.toNat <= 0xFC
11     . simp [hb] at h; rcases h with <rfl, rfl>; simp
12     . simp [hb] at h
13     by_cases hfd : b.toNat = 0xFD
14     ...

```

SortedList 7, SortedSet 7) and every one of them unattempted rather than rejected. The instance is a Lean port of the python-sortedcontainers library, and its reference implementation follows the Python original in using binary search over an index range:

```

1  def bisectLeftGo (lst : List a) (val : a) (lo hi : Nat) : Nat :=
2    ...
3    if ... then bisectLeftGo lst val (mid + 1) hi
4    else bisectLeftGo lst val lo mid
5  termination_by hi - lo

```

Proving anything about bisectLeftGo requires an invariant tying the lo/hi window to the sortedness of the enclosing list, maintained across a recursion whose measure is hi - lo rather than the structure of the list. That invariant is what the 25 unattempted proof-only specifications need, and no configuration supplies it.

In code-and-proof the agent avoids the invariant entirely by choosing a different algorithm. It adds 110 lines to Impl/SortedList.lean implementing insertion-based sorting and a linear scan in place of bisection:

```

1  def linearBisectLeft {a : Type} [Ord a] (value : a) : List a -> Nat
2    | [] => 0
3    | x :: xs => if ordLt x value then linearBisectLeft value xs + 1 else 0
4
5  def insertOrd {a : Type} [Ord a] (value : a) : List a -> List a
6    ...
7  def sortOrd {a : Type} [Ord a] : List a -> List a
8    | x :: xs => insertOrd x (sortOrd xs)

```

Both definitions recurse on the list constructor, so every specification about them is provable by induction on the list with no auxiliary window invariant. The substitution costs asymptotic efficiency, giving $O(n)$ lookup instead of $O(\log n)$ and $O(n^2)$ construction instead of $O(n \log n)$, so the agent’s implementation is not a drop-in replacement for the library it ports. It nonetheless satisfies every scored specification, because the specifications constrain the input–output behavior of the container rather than its complexity.

This illustrates a limitation of our metric that Appendix D discusses in general terms. A full solve in code-and-proof certifies that the submitted implementation meets the submitted specifications, but it does not certify that the implementation preserves properties the specifications do not mention. Complexity is the most consequential such property for a data-structure library, and it is exactly what an agent will trade away when the proof obligation is the binding constraint. Specifying cost behavior formally is possible in principle but requires a cost model the source repositories do not provide, so we note the gap rather than close it.